AIMZ Egypt

Cloudflare → AWS Migration Runbook

How to launch the platform and push updates to AWS

Infrastructure: AWS CDK (TypeScript)
Compute: EC2 + ALB Data: RDS PostgreSQL + ElastiCache Redis
Storage/CDN: S3 + CloudFront

Target scale: **~500 concurrent users**

Generated for the `aimz` monorepo — `infra/aws-cdk`

Contents

1 Read this first: security

An AWS access key was shared in plaintext during preparation of this migration. Treat it as compromised. Before anything else: sign in to the AWS console, open IAM → Users → Security credentials, deactivate and delete that access key, then create a fresh one. Never paste long-lived keys into chats, tickets, or source files.

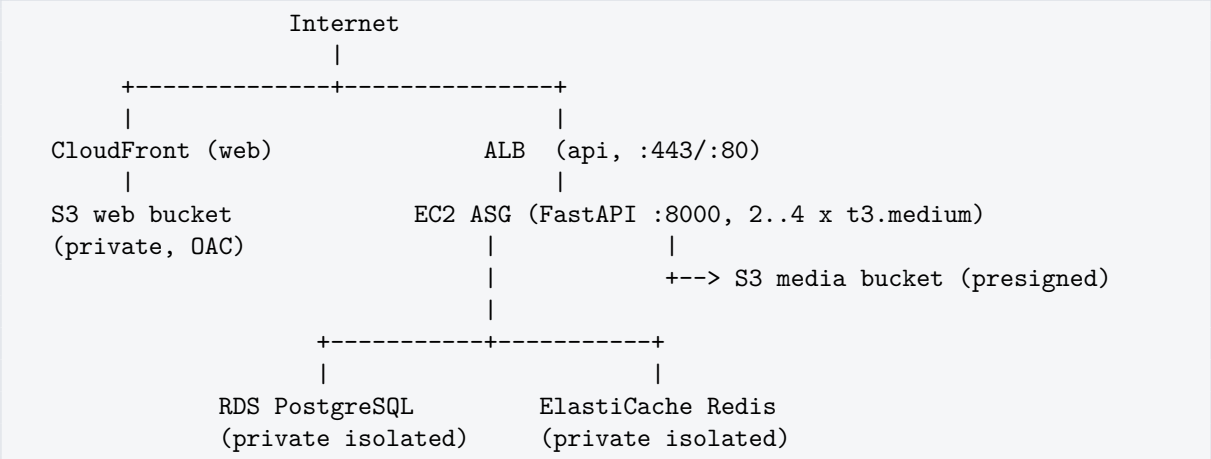
This project never stores AWS credentials in code. The CDK and the helper scripts authenticate through your local AWS CLI profile (`aws configure` or SSO). Application secrets (database password, JWT secret, admin password, SMTP) live only in **AWS Secrets Manager** and are pulled by the API instances at boot.

2 What this migration does

The live API today runs on Cloudflare Workers (Hono) backed by D1 (SQLite) and R2, with the web app on Cloudflare Pages. This migration moves the platform onto a conventional, horizontally-scalable AWS stack driven entirely by the FastAPI backend in `backend/`, which already speaks PostgreSQL and S3.

Concern	Cloudflare (before)	AWS (after)
API compute	Workers (Hono)	EC2 Auto Scaling Group + ALB
Database	D1 (SQLite)	RDS PostgreSQL
Cache	— (none)	ElastiCache Redis
Object storage	R2	S3 (private, presigned)
Web hosting	Pages	S3 + CloudFront (OAC)
Secrets	Worker vars	Secrets Manager
Image registry	—	ECR (via CDK assets)

2.1 Architecture at a glance



All data services live in private, isolated subnets. Only the ALB and CloudFront are internet-facing. The API tier reaches the internet for image pulls and AWS APIs through a single NAT gateway.

3 Prerequisites

- An AWS account and an IAM identity with administrator (or equivalently broad) permissions for the initial bootstrap.
- Local tools: `aws` CLI v2, Node.js 20+, Docker (running), `jq`, and `git`.
- AWS CLI configured for the target account and region:

```
aws configure           # or: aws configure sso
aws sts get-caller-identity   # confirm you are the right account
export AWS_REGION=eu-west-1    # pick your region once
```

- Node dependencies installed for the CDK app:

```
cd infra/aws-cdk
npm install
```

4 One-time bootstrap

CDK needs a small bootstrap stack (an asset bucket, ECR repo, and roles) in each account/region before the first deploy:

```
cd infra/aws-cdk
npx cdk bootstrap aws://<ACCOUNT_ID>/<REGION>
```

Run this once per account/region. It is safe to re-run.

5 Configuration knobs

Everything tunable is passed as CDK *context* (`-c key=value`) or an environment variable, with defaults sized for ∼500 concurrent users.

Context key	Default	Purpose
`env`	`production`	Logical environment / stack suffix
`apiInstanceType`	`t3.medium`	EC2 class for API instances
`apiMinCapacity`	2	Min instances (HA across 2 AZs)
`apiMaxCapacity`	4	Max instances (CPU target 60%)
`dbInstanceClass`	`t3.medium`	RDS instance class
`dbMultiAz`	`false`	RDS Multi-AZ standby (set true for prod HA)
`redisNodeType`	`cache.t4g.small`	ElastiCache node class
`webDomainName`	(none)	Custom domain for CloudFront
`webCertificateArn`	(none)	ACM cert (us-east-1) for the web domain
`apiCertificateArn`	(none)	ACM cert (stack region) for ALB HTTPS

5.1 Sizing note for 500 concurrent users

The defaults give two `t3.medium` API nodes (2 vCPU / 4 GiB each) behind an ALB, auto-scaling to four at 60% CPU. A scores/roster workload is bursty and read-heavy, so a `db.t3.medium` with the Redis cache in front absorbs 500 concurrent users comfortably. For sustained peak traffic, raise `apiMaxCapacity`, move the database to `db.m6g.large`, and set `dbMultiAz=true`.

6 Launch (first deploy)

From `infra/aws-cdk`:

```
# 1. Review what will be created (optional but recommended)
npx cdk diff -c env=production

# 2. Create everything. CDK builds the backend Docker image, pushes it to ECR,
#    and stands up VPC, RDS, Redis, S3, CloudFront, ALB and the ASG.
npx cdk deploy -c env=production --require-approval never
```

The first deploy takes roughly 20–30 minutes (RDS and CloudFront dominate). At the end, CDK prints the stack outputs; the ones you need:

Output	Use
`ApiUrl`	Public API base URL (goes into the mobile build)
`WebUrl`	CloudFront URL of the web app
`AsgName`	Used by the migration/redeploy scripts
`DbEndpoint`, `RedisEndpoint`	Data endpoints (informational)

6.1 Post-deploy steps (run once)

1. **Set the admin password.** Hosted environments refuse the placeholder value:

```
./scripts/set-admin-password.sh production 'a-strong-admin-password'
    admin@yourdomain.com
./scripts/redeploy-api.sh production   # recycle instances to pick it up
```

2. **Run database migrations + seed.** This applies Alembic migrations and creates the admin account and initial invite. It runs inside one instance's container over SSM (no SSH), so it never races the ASG:

```
./scripts/run-migrations.sh production
```

3. **Publish the web app.** Builds the Expo web export against `ApiUrl` and pushes it to the CloudFront bucket:

```
./scripts/deploy-web.sh production
```

4. **Verify.**

```
curl -s "$(aws cloudformation describe-stacks --stack-name Aimz-production \
  --query "Stacks[0].Outputs[?OutputKey=='ApiUrl'].OutputValue" --output
    text)/api/v1/health"
# -> {"status":"ok","service":"aimz-api",...}
```

7 Pushing updates

7.1 Backend / API changes

Any change under `backend/` ships as a new container image:

```
cd infra/aws-cdk
npx cdk deploy -c env=production     # rebuilds + pushes the image to ECR
./scripts/redeploy-api.sh production # rolling instance refresh onto the new image
# If the change added migrations:
./scripts/run-migrations.sh production
```

7.2 Web / mobile-web changes

```
cd infra/aws-cdk
./scripts/deploy-web.sh production    # rebuild export, sync to S3, invalidate CDN
```

7.3 Infrastructure changes

Edit `lib/aimz-stack.ts`, then:

```
npx cdk diff -c env=production        # preview
npx cdk deploy -c env=production      # apply
```

8 Operations

Shell into an instance
`aws ssm start-session -target <instance-id>` (no SSH keys, no open ports). Then `docker logs -f aimz-api`.

API logs
The container logs to the instance; for aggregation, ship `docker logs` to CloudWatch (future enhancement) or use the Session Manager shell above.

Scaling
The ASG target-tracks CPU at 60%. Change `apiMin/MaxCapacity` and redeploy, or edit the ASG in the console for an immediate bump.

Database backups
RDS automated backups retain 7 days. Production uses a final snapshot on stack deletion and deletion protection.

Secrets rotation
Update the value in Secrets Manager (or via `set-admin-password.sh`) and run `redeploy-api.sh` so instances re-read it at boot.

9 Cost sketch (order of magnitude)

Rough monthly, on-demand, single region — confirm against the AWS pricing calculator for your region:

Component	Est. / month (USD)
2× t3.medium EC2 (API)	~60
ALB	~20
RDS db.t3.medium (single-AZ) + 50GB	~70
ElastiCache cache.t4g.small	~25
NAT gateway	~35
S3 + CloudFront (light)	~10
Total	**~220**

Multi-AZ RDS and a Redis replica roughly double their respective lines and are recommended before you rely on this in production.

10 Teardown

Destroys data. In production the database takes a final snapshot and buckets are retained; elsewhere everything is removed.

```
cd infra/aws-cdk
npx cdk destroy -c env=production
```

11 Remaining work (staged plan)

This runbook covers **Stage 1**: the AWS infrastructure and deployment tooling. Two stages remain before the AWS deployment fully replaces Cloudflare:

1. **Stage 2 — FastAPI feature parity.** The Cloudflare Worker is ahead of the FastAPI backend by roughly 15 migrations and eight route groups (stats, knockout, training, announcements, assignments, roster, calendar, audit). These must be ported into `backend/` so the AWS API matches the contract the mobile app expects. The Redis cache provisioned here is then wired into the hot read paths.
2. **Stage 3 — Cutover.** Point `EXPO_PUBLIC_API_URL` at the ALB, migrate live data from D1 into RDS, publish the web app to CloudFront, ship the mobile build, and retire the Workers/Pages/D1/R2 resources.

Infrastructure as code: `infra/aws-cdk/lib/aimz-stack.ts` — one `cdk deploy` stands up the whole platform.